# YAML-Grundlagen

Lernmaterial: CC BY 4.0 (siehe `LICENSE-CONTENT.txt`).

## Ziel

Du kannst die Grundstruktur einer YAML-Datei lesen, einfache Änderungen an einem GitHub-Actions-Workflow vornehmen und typische Einrückungsfehler erkennen.

## Wo begegnet dir YAML?

YAML ist ein Textformat, mit dem Daten übersichtlich beschrieben werden. Programme lesen diese Daten und verwenden sie als Konfiguration. In diesem Kurs beschreiben die Dateien unter `.github/workflows/` damit, wann ein Workflow startet und welche Jobs und Schritte er ausführt.

**Datei prüfen:** Öffne die Vorlage `.github/workflows/01-hallo-linux.yml` im Kurs-Repository oder später die gleichnamige Datei in deinem Lern-Repository. Der folgende Ausschnitt zeigt den Anfang des Workflows aus Teil 1:

```yaml
name: 01 - Hallo Linux

on:
  workflow_dispatch:

jobs:
  erkunden:
    runs-on: ubuntu-latest
    steps:
      - name: Begrüßung
        run: echo "Hallo aus dem Linux-Runner!"
      - name: Wer bin ich?
        run: whoami
```

YAML legt hier die Struktur und Einstellungen fest. GitHub Actions deutet bestimmte Schlüssel wie `name`, `on`, `jobs` und `steps` als Workflow-Konfiguration. Der Inhalt von `run` ist dagegen ein Kommando, das später in der Shell des Runners ausgeführt wird.

### Schlüssel und Werte

Ein **Schlüssel (key)** benennt eine Einstellung; der dazugehörige **Wert (value)** gibt ihren Inhalt an. Ein Doppelpunkt trennt Schlüssel und Wert:

```
name: 01 - Hallo Linux
runs-on: ubuntu-latest
```

Hier heißen die Schlüssel `name` und `runs-on`. Ihre Werte sind `01 - Hallo Linux` und `ubuntu-latest`. In GitHub-Actions-Dateien sind zulässige Schlüssel und ihre Bedeutung durch die Workflow-Syntax von GitHub Actions festgelegt.

**Aufgabe:** Lies den Ausschnitt aus `01-hallo-linux.yml` und finde die Werte zu `name` und `runs-on`.

**Ausführung:** Öffne die Datei im Browser oder in einem Texteditor und suche die beiden Schlüssel.

**Kontrolle:** `name` bezeichnet den Workflow als „01 - Hallo Linux“. `runs-on` gibt die Runner-Umgebung `ubuntu-latest` an.

### Einrückung zeigt Zugehörigkeit

Einrückung macht sichtbar, welche Angaben zu einem übergeordneten Schlüssel gehören. In diesem Kurs rücken wir jede Ebene um zwei Leerzeichen ein und verwenden **keine** Tabulatoren. Achte darauf, gleichrangige Einträge auf derselben Ebene auszurichten.

```
jobs:
  erkunden:
    runs-on: ubuntu-latest
    steps:
      - name: Begrüßung
        run: echo "Hallo aus dem Linux-Runner!"
```

`erkunden` gehört zu `jobs`; `runs-on` und `steps` gehören zum Job `erkunden`. `name` und `run` gehören zum ersten Schritt.

**Aufgabe:** Zeichne die Zugehörigkeit der Schlüssel im Ausschnitt als Baum oder markiere jede Einrückungsebene mit einer anderen Farbe.

**Ausführung:** Gehe von links nach rechts: Ein weiter eingerückter Eintrag gehört zum passenden übergeordneten Eintrag.

**Kontrolle:** Der Baum lautet `jobs → erkunden → steps → Schritte`; `runs-on` gehört ebenfalls direkt zu `erkunden`.

### Listen

Eine **Liste** enthält mehrere Einträge. In YAML beginnen Einträge einer eingerückten Liste mit einem Bindestrich und einem Leerzeichen. In einem Workflow sind die Schritte eines Jobs eine Liste:

```
steps:
  - name: Begrüßung
    run: echo "Hallo aus dem Linux-Runner!"
  - name: Wer bin ich?
    run: whoami
```

Jeder Bindestrich beginnt einen Schritt. `name` und `run` sind Einstellungen dieses Schritts.

**Aufgabe:** Zähle die Schritte im Ausschnitt und nenne für jeden den Namen.

**Ausführung:** Zähle die Einträge, die mit `- name:` beginnen.

**Kontrolle:** Es sind zwei Schritte: „Begrüßung“ und „Wer bin ich?“.

### Verschachtelte Angaben

Eine **verschachtelte Struktur** enthält Daten innerhalb anderer Daten. Im Workflow gehören Schritte zu einem Job und Jobs zum Workflow. Ihre Einrückung zeigt diese Ebenen. Du kannst den Aufbau in Teil 2 bei `needs` und Job-Ausgaben sowie in Teil 3 bei Matrizen wiedererkennen.

**Aufgabe:** Finde in `.github/workflows/05-mehrere-jobs.yml` den Schlüssel `jobs` und verfolge die Einrückung bis zu `steps`.

**Ausführung:** Öffne die Datei in deinem Lern-Repository und vergleiche sie mit dem Grundmuster aus `01-hallo-linux.yml`.

**Kontrolle:** `steps` steht innerhalb eines Jobs unter `jobs`; die Schritte selbst sind Listeneinträge.

### Textwerte und Anführungszeichen

Werte können als normaler Text geschrieben oder in einfache beziehungsweise doppelte Anführungszeichen gesetzt werden. Anführungszeichen sind nützlich, wenn ein Wert Zeichen enthält, die YAML oder GitHub Actions eine besondere Bedeutung geben könnten. Die genaue Interpretation kann außerdem vom verwendeten YAML-Parser und vom Kontext in GitHub Actions abhängen. Folge deshalb dem Stil des jeweiligen Beispiels und prüfe Workflow-Angaben anhand der GitHub-Dokumentation.

In den Workflows dieses Kurses stehen Pfadmuster und Cron-Ausdrücke zum Beispiel in Anführungszeichen. Ändere solche Werte nicht, ohne die dazugehörige Erklärung in der Lektion zu lesen.

### Mehrzeilige Werte: `run: |`

Ein senkrechter Strich (`|`) beginnt in YAML einen mehrzeiligen Textblock. In einem Workflow steht darunter eingerückter Shell-Code, den GitHub Actions als Inhalt des `run`-Schritts ausführt:

```
jobs:
  dateien:
    steps:
      - name: Ordner und Dateien anlegen
        run: |
          mkdir -p arbeit
          touch arbeit/notizen.txt
          echo "Erste Notiz" > arbeit/notizen.txt
          ls -l arbeit
```

Die Zeilen unter `run: |` sind keine YAML-Schlüssel. YAML grenzt den Textblock ab; die Shell verarbeitet anschließend die darin enthaltenen Befehle. Der Ausschnitt entspricht dem Schritt in `.github/workflows/02-dateien.yml`.

**Aufgabe:** Zähle in dem Ausschnitt die Shell-Befehle unter `run: |`.

**Ausführung:** Öffne die Datei und unterscheide den Schlüssel `run` vom eingerückten Inhalt.

**Kontrolle:** Der mehrzeilige Inhalt gehört zu einem einzelnen `run`-Schritt und enthält vier Shell-Befehle.

### Kommentare

Ein YAML-Kommentar beginnt mit `#` außerhalb eines Textwerts und reicht bis zum Zeilenende. Kommentare dienen Menschen als Hinweis; sie sind keine Workflow-Einstellungen und werden nicht als Workflow-Schritt ausgeführt.

### Fehler finden und prüfen

Ein YAML-Fehler kann verhindern, dass GitHub Actions eine Workflow-Datei korrekt verarbeitet. Fehlerhafte Einrückung, ein fehlender Doppelpunkt oder falsch platzierte Listeneinträge sind typische Ursachen. Eine Workflow-Ausführung ist kein allgemeiner YAML-Prüfer: Wenn eine Datei nicht eingelesen werden kann, kann der Lauf bereits vor dem Start der Jobs scheitern.

**Aufgabe:** Prüfe eine Änderung an einer Workflow-Datei vor dem Start des Workflows. Ändere für diese Übung nur einen vorhandenen Schritt, zum Beispiel seinen `name`-Wert.

**Ausführung:** Bearbeite die Datei im Lern-Repository, achte auf Einrückung und vergleiche die Struktur mit dem unveränderten Nachbarbeispiel. Committe die Änderung nach den Schritten in Git-Grundlagen. Starte den Workflow über **Actions → Run workflow**.

**Kontrolle:** Prüfe, ob GitHub den Workflow anzeigt, ob der Lauf startet und ob der Schritt im Protokoll den geänderten Namen trägt. Diese Kontrollen prüfen unterschiedliche Dinge: Eine angezeigte Datei allein beweist nicht, dass der Workflow erfolgreich ausgeführt wurde.

Bei einem Workflow mit `workflow_dispatch` muss die Workflow-Datei auf dem Standardbranch liegen, damit GitHub den manuellen Start anbietet. Beim Start kannst du den Branch auswählen, dessen Dateien ausgeführt werden sollen. Weitere Hinweise dazu stehen in Teil 1.

### Typische Fehler

- **GitHub verarbeitet die Workflow-Datei nicht:** Prüfe Einrückung, Doppelpunkte und Position der Listeneinträge.
- **Ein Schritt fehlt oder steht an der falschen Stelle:** Prüfe, ob Bindestrich und Einrückung den Schritt unter `steps` des gewünschten Jobs einordnen.
- **Shell-Befehl erscheint als YAML-Einstellung:** Lies `run` als Workflow-Schlüssel und die eingerückten Zeilen darunter als Shell-Code im mehrzeiligen Textblock.
- **Run workflow wird nicht angeboten:** Prüfe, ob die Workflow-Datei auf dem Standardbranch liegt und einen manuellen Trigger enthält.

### Weiterlernen

Die Beispiele in diesem Modul verwenden Ausschnitte aus den Workflow-Dateien des Kurses. Für die vollständige Syntax von GitHub Actions gelten die offiziellen Workflow-Syntaxregeln[1]. Die allgemeine YAML-Sprache ist in der YAML-Spezifikation 1.2.2[2] beschrieben. GitHub-Actions-Workflows verwenden YAML, ergänzen aber eigene Schlüssel und Regeln; die allgemeine YAML-Spezifikation allein erklärt deshalb nicht die gesamte Workflow-Konfiguration.

[1] https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions
[2] https://yaml.org/spec/1.2.2/